Sessions 16–18:
Java File Handling
& Serialization

Byte Streams • Character Streams • Object Serialization

Course module focusing on Java I/O fundamentals, best practices, and persistence.

Session 16 — Objectives

Byte Streams

- Understand binary file handling in Java
- Learn InputStream and OutputStream hierarchy
- Read and write files using byte streams
- Handle file exceptions and resource management

What Are Byte Streams?

Concept & Syntax Overview

- Process **raw binary data** (8-bit bytes)
- Suitable for **images, audio, PDFs**, archives
- Operate **byte-by-byte**; no character decoding
- Core classes: **InputStream, OutputStream**
- File-specific: **FileInputStream, FileOutputStream**

Example: Reading a Binary File

```
// Reading an image file byte-by-byte
try (FileInputStream fin = new
FileInputStream("image.png")) {
  int byteData;
  // read() returns -1 when EOF is reached
  while ((byteData = fin.read()) != -1) {
    // Process the raw 8-bit binary data
    processByte(byteData);
  }
} catch (IOException e) {
  e.printStackTrace();
}
```

InputStream & OutputStream

Class Hierarchy & Decorator Pattern

- Two abstract base classes: **InputStream** & **OutputStream**
- Concrete nodes: **FileInputStream, FileOutputStream**
- **Filter streams** add functionality via Decorator pattern
- **BufferedXxxStream**: Vastly improves I/O performance
- Chain multiple streams to combine behaviors seamlessly

Example: Stream Decoration

```
// Chaining streams for buffered primitive I/O
try (DataOutputStream dout = new DataOutputStream(
new BufferedOutputStream(
new FileOutputStream("data.bin")))) {

// Now we can write primitives efficiently
dout.writeInt(42);
dout.writeDouble(3.14159);
dout.writeBoolean(true);

} catch (IOException e) {
e.printStackTrace();
}
```

Reading with FileInputStream

Byte-by-Byte vs. Buffered Reading

- Reads **raw bytes** sequentially from a file in the file system.
- The **read()** method returns **-1** when End-Of-File (EOF) is reached.
- Reading **byte-by-byte** directly from disk is highly inefficient.
- **BufferedInputStream** reads chunks into memory, vastly improving speed.
- Use **try-with-resources** to guarantee automatic stream closure.

</> FileInputStream Examples

```
// 1. Basic Byte-by-Byte (Slower)
try (FileInputStream fin = new FileInputStream("data.bin"))
{
int b;
while ((b = fin.read()) != -1) {
// Process single byte 'b'
}
}
// 2. Buffered Reading (Faster & Recommended)
try (BufferedInputStream bin = new BufferedInputStream(
new FileInputStream("data.bin"))) {
byte[] buffer = new byte[8192]; // 8KB buffer
int bytesRead;
while ((bytesRead = bin.read(buffer)) != -1) {
// Process array chunk: buffer[0] to buffer[bytesRead-1]
}
}
```

Writing with FileOutputStream

Concept & Syntax Overview

- Used for writing streams of **raw bytes**
- Convert Strings to byte arrays via **getBytes()**
- Use **write(byte[])** to output data to the file
- Append mode: pass **true** in the constructor
- Requires proper resource **closure (TWR)**

Example: Basic Write & Append

```
String text = "Hello Java\n";
byte[] data = text.getBytes(StandardCharsets.UTF_8);

// 1. Standard Write (Overwrites existing file)
try (FileOutputStream fout = new
FileOutputStream("out.bin")) {
    fout.write(data);
}

// 2. Append Mode (Adds to existing file)
try (FileOutputStream fout = new
FileOutputStream("out.bin", true)) {
    fout.write(data);
}
```

Byte Streams: Best Practices

Resource Management & Exceptions

- Always use **try-with-resources** for automatic resource closing
- Prefer **BufferedInputStream** / **BufferedOutputStream** for large files
- Be mindful of binary vs text; do not cast **byte** to **char** blindly
- Properly handle **FileNotFoundException** and **IOException**
- Avoid reading one byte at a time in tight loops without buffering

Hands-on Exercises

Byte Streams

- ☐ Read a file and print total byte count
- ☐ Copy a file (source → destination) efficiently with 8 KB buffer
- ☐ Measure time with and without buffering
- 🏆 **Challenge:** Compute SHA-256 of a file using MessageDigest

Session 17 — Objectives

Character Streams

- Handle text files efficiently
- Understand Reader and Writer classes
- Use FileReader, FileWriter, and BufferedReader/Writer
- Choose between byte vs character streams correctly

What Are Character Streams?

Text Data & Reader/Writer Hierarchy

- Process **text data** (Unicode characters)
- Perform **decoding/encoding** under the hood
- Typical files: **.txt, .csv, .log**
- Core classes: **Reader, Writer**
- File-specific: **FileReader, FileWriter**

</> Example: Reader & Writer

```
// Handling text files with auto encoding
try (Reader reader = new FileReader("in.txt");
Writer writer = new FileWriter("out.txt")) {
int charData;
// Reads Unicode characters (16-bit) automatically
while ((charData = reader.read()) != -1) {
writer.write(charData);
}
} catch (IOException e) {
e.printStackTrace();
}
```

FileReader — Simple Read

Reading Characters & Managing Charsets

- Reads characters directly from text files (**.txt, .csv**)
- Uses platform's **default charset** implicitly

- Can cause **encoding issues** across different OS (Windows vs Linux)
- Recommended: Use **InputStreamReader** with explicit charset (UTF-8)

Example: Reading Text Files

```
// Basic read (Uses Default Charset)
try (FileReader fr = new FileReader("notes.txt")) {
int ch;
while ((ch = fr.read()) != -1) {
    System.out.print((char) ch);
}
} catch (IOException e) { ... }

// Explicit Charset (Recommended for Portability)
try (InputStreamReader isr = new InputStreamReader(
new FileInputStream("notes.txt"),
StandardCharsets.UTF_8)) {
   int ch;
   while ((ch = isr.read()) != -1) {
       System.out.print((char) ch);
  }
} catch (IOException e) { ... }
```

BufferedReader — Line by Line

Character Streams in Action

- Reads text line-by-line using **readLine()**
- Returns **null** when end of file (EOF) is reached
- **Buffers characters** to minimize I/O operations
- Ideal for parsing **CSV, logs, and config** files

Example: Reading Line-by-Line

```
// Read text file line-by-line
try (BufferedReader br = new BufferedReader(
new FileReader("notes.txt"))) {
String line;
// readLine() returns null when EOF is reached
while ((line = br.readLine()) != null) {
// Process the full line of text
System.out.println(line);
}
} catch (IOException e) {
e.printStackTrace();
}
```

FileWriter — Write & Append

Character Streams Output

- Writes **streams of characters** to a file
- Convenient methods for **String** and **char[]**
- Default behavior: **Overwrites** existing files
- Append mode: Pass **true** as second argument
- Must be **closed/flushed** to ensure data is saved

Example: Writing and Appending

```
// 1. Write mode (Overwrites existing file)
try (FileWriter fw = new FileWriter("report.txt")) {
fw.write("Learning Java File Handling\n");
} catch (IOException e) {
e.printStackTrace();
}

// 2. Append mode (Adds to the end of file)
try (FileWriter fw = new FileWriter("report.txt", true)) {
fw.write("More lines appended here...\n");
} catch (IOException e) {
e.printStackTrace();
}
```

Byte vs Character Streams

Choosing the correct stream type is crucial for performance, proper data handling, and maintaining file integrity.

Feature	Byte Streams	Character Streams
Data Type	Binary data (8-bit bytes)	Text data (16-bit Unicode characters)
Base Classes	InputStream / OutputStream	Reader / Writer
Typical Usage	Images, audio, archives (.zip), executables	Log files, .csv, .txt, JSON, XML
Encoding	None (reads and writes raw bytes)	Charset-aware (handles decoding/encoding)
Common Classes	FileInputStream, FileOutputStream	FileReader, FileWriter, BufferedReader

Hands-on Exercises

Character Streams

- ☐ Read a text file line-by-line and print with line numbers
- ☐ Count lines, words, and characters
- ☐ Append a summary line at the end of the file
- 🏆 **Challenge:** Convert file encoding (e.g., ISO-8859-1 → UTF-8)

Session 18 — Objectives

Serialization & Deserialization

- Understand Java object serialization
- Persist and retrieve object state from files
- Implement Serializable correctly (IDs, transient keyword)
- Handle I/O and class versioning issues

What Is Serialization?

Object Persistence & Byte Streams

- Converts **objects into a byte stream**
- Enables **persistence to disk** and network transfer
- Paired with **deserialization** to reconstruct objects
- Use when storing state or sending over **sockets/RMI**
- Beware of **versioning and security** considerations

</> Serialization & Deserialization Flow

```
// 1. Serialization (Write to Stream)
try (ObjectOutputStream out = new ObjectOutputStream(
new FileOutputStream("data.obj"))) {
out.writeObject(myObject); // Object -> Bytes
}

// 2. Deserialization (Read from Stream)
try (ObjectInputStream in = new ObjectInputStream(
new FileInputStream("data.obj"))) {
Object obj = in.readObject(); // Bytes -> Object
}
```

Serializable Interface

Impementing the Model Class

- Implement **java.io.Serializable**
- **Marker interface** (contains no methods)
- Requires **serialVersionUID** for versioning
- State of **instance variables** is saved automatically

Example: Student.java

```
import java.io.Serializable;

public class Student implements Serializable {
private static final long serialVersionUID = 1L; //
version control

String name;
int marks;

public Student(String name, int marks) {
this.name = name;
this.marks = marks;
}

@Override
public String toString() {
return name + ": " + marks;
}
}
```

ObjectOutputStream

Serializing Objects into Byte Streams

- Writes objects directly to an **OutputStream**
- Target objects MUST implement **Serializable**
- Use the **writeObject(Object)** method
- Automatically serializes the entire **object graph**

Example: Writing Object to File

```
// 1. Create instance of Serializable class
Student s = new Student("Amit", 90);

// 2. Open streams via try-with-resources
try (ObjectOutputStream out = new ObjectOutputStream(
new FileOutputStream("student.dat"))) {

// 3. Serialize object into file
out.writeObject(s);

} catch (IOException e) {
e.printStackTrace();
}
```

ObjectInputStream — Deserialize

Reconstructing Objects from Files

- Reads **byte streams** and reconstructs Java objects
- Requires corresponding **class definition** to exist in the classpath
- Requires explicit casting: `(Student) in.readObject()`
- Must handle **ClassNotFoundException**
- Restores entire **object graphs** automatically

Example: Deserializing an Object

```
// Reading a serialized object from a file
try (ObjectInputStream in = new
ObjectInputStream(

new FileInputStream("student.dat"))) {
// Read and cast the object
Student s = (Student) in.readObject();
System.out.println(s.name + ": " + s.marks);
} catch (IOException | ClassNotFoundException e)
{
e.printStackTrace();
}
```

KEY TERM

serialVersionUID

Version Control in Serialization

Concept & Purpose

- Unique **long** constant identifying the specific version of a serialized class.
- Mismatch causes **InvalidClassException** during the deserialization process.
- Define explicitly to ensure proper cross-release compatibility.
- Change value when making incompatible structural changes to the class.

</> Implementation Example

```
import java.io.Serializable;

public class Student implements Serializable {

    // Explicit version control ID
private static final long serialVersionUID = 1L;

    private String name;
    private int marks;
}
```

KEY TERM

transient Keyword

Exclude Sensitive Fields from Serialization

Concept & Purpose

- Fields marked **transient** are ignored during the serialization process.
- Useful for protecting sensitive data like passwords or security tokens.
- Ideal for caches, temporary states, or derived calculated fields.
- Note: **static** fields are never serialized as they belong to the class level.

</> Implementation Example

```
import java.io.Serializable;

class Account implements Serializable {

    private static final long serialVersionUID = 1L;

    String username;

    // Excluded from serialization
transient String password;
}
```

Hands-on & Summary

Serialization Exercises & Course Recap

- ☐ Create a Student object, serialize to a file, then deserialize and print
- ☐ Test `transient` fields and change `serialVersionUID` to observe exceptions

Summary: Sessions 16–18

- **Byte Streams:** `FileInputStream` / `FileOutputStream`, buffering, binary data exceptions
- **Character Streams:** `FileReader` / `BufferedReader` / `FileWriter`, text encoding awareness
- **Serialization:** `Serializable`, `ObjectOutputStream`, version control, sensitive fields